

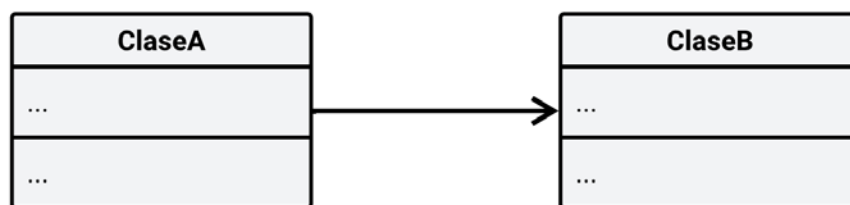
ASOCIACIÓN

- Una clase “**conoce**” a otra clase.
- Una clase se construye a partir de otra/s clase/s.
- El tiempo de vida de una clase no depende de la otra.
- Es una relación más fuerte que **DEPENDENCIA**, pero más débil que **AGREGACIÓN**.

Formato:

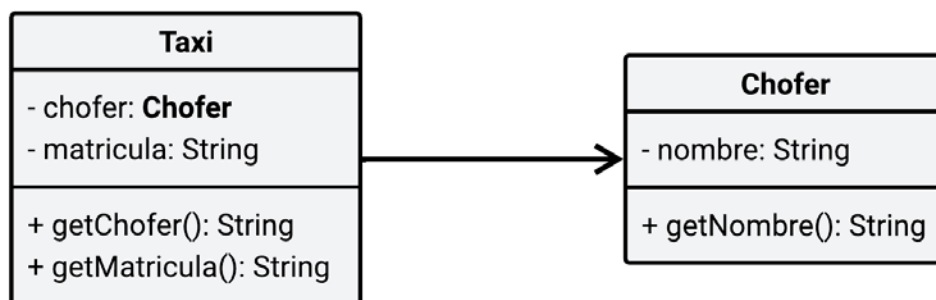
- Se representa con una flecha continua.
- El sentido de la flecha es unidireccional:
 - o Origen: la clase que conoce (a otra clase).
 - o Destino: la clase que es conocida por la primera.

Especificación:



- ClaseA **está asociada** o **conoce** a ClaseB.
- ClaseA sabe de la existencia de ClaseB, pero ClaseB no sabe que existe ClaseA.
- ClaseA depende de ClaseB → todo cambio en ClaseB podrá afectar a ClaseA.
- ClaseA tendrá como atributo una instancia de ClaseB.
- ClaseA, usando sus métodos, podrá acceder a los atributos y métodos de ClaseB.

Ejemplo Práctico:



- La clase Taxi tiene un atributo `matricula`.
- La clase Chofer tiene un atributo `nombre`.
- Cada instancia de la clase Taxi tendrá asociado un Chofer.
- Taxi necesita acceder a los atributos de Chofer (a su `nombre`, por ejemplo).

DEPENDENCIA

- Una clase “usa” a otra clase.
- Es la relación más débil, comparada con las demás relaciones.

Formato:

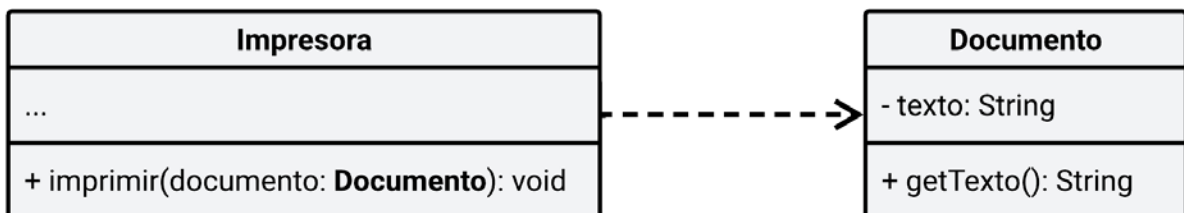
- Se representa con una flecha punteada.
- El sentido de la flecha es unidireccional:
 - o Origen: la clase que usa (a otra clase).
 - o Destino: la clase que es usada por la primera.

Especificación:



- ClaseA usa a ClaseB.
- ClaseA sabe de la existencia de ClaseB, pero ClaseB no sabe que existe ClaseA.
- ClaseA depende de ClaseB → todo cambio en ClaseB podrá afectar a ClaseA.
- ClaseA hace uso de ClaseB:
 - o Instanciándola; o bien
 - o Recibiéndola como parámetro de entrada en uno de sus métodos.

Ejemplo Práctico:



- La clase Impresora tiene un método imprimir.
- La clase Documento tiene un atributo texto.
- La clase Impresora se encarga de imprimir un Documento.

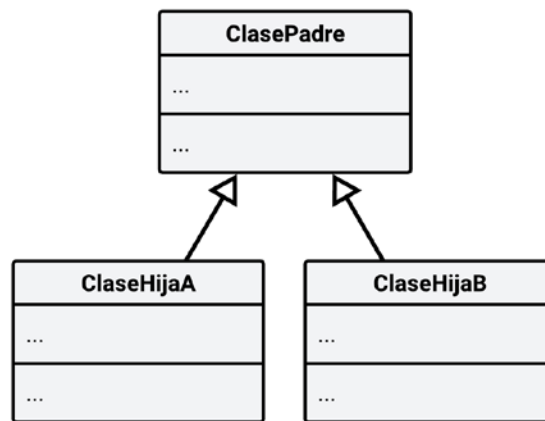
GENERALIZACIÓN

- Una clase particular (o clase hija o subclase) “**hereda**” los atributos, métodos y relaciones de una clase más general (o clase padre o superclase).
- Si una clase hija, tras heredar, necesita redefinir el comportamiento de su clase padre, se deben escribir los nombres de los métodos que redefine.
- La idea es minimizar las repeticiones de código.

Formato:

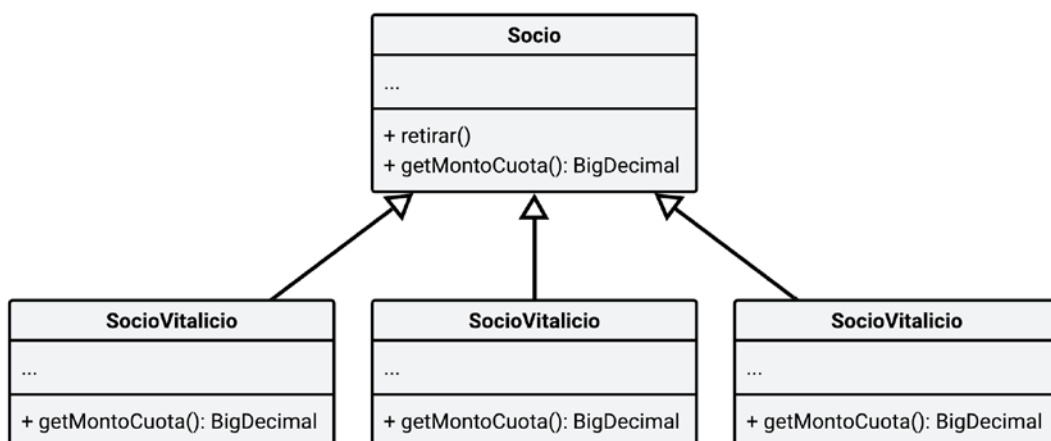
- Se representa con una flecha continua, con terminación en un triángulo cerrado.
- El sentido de la flecha es unidireccional:
 - o Origen: la clase hija que **hereda** de la clase padre.
 - o Destino (triángulo): la clase padre de la que heredan su/s clase/s hija/s.

Especificación:



- ClaseHijaA y ClaseHijaB **heredan** de ClasePadre.

Ejemplo Práctico:



- La clase Socio tiene un método getMontoCuota.
- Las subclases SocioVitalicio, SocioEspecial y SocioComún heredan de la clase Socio y redefinen el comportamiento del método getMontoCuota, cada una en forma independiente.

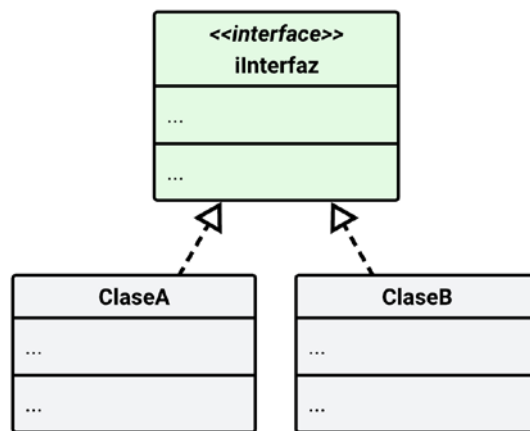
REALIZACIÓN

- Se establece entre una clase y una interfaz.
- Una clase “**implementa**” todos los métodos que defina la interfaz.
- La idea es minimizar las repeticiones de código.

Formato:

- Se representa con una flecha punteada, con terminación en un triángulo cerrado.
- El sentido de la flecha es unidireccional:
 - o Origen: la clase que implementa la interfaz.
 - o Destino (triángulo): la interfaz desde donde se implementan las clases.

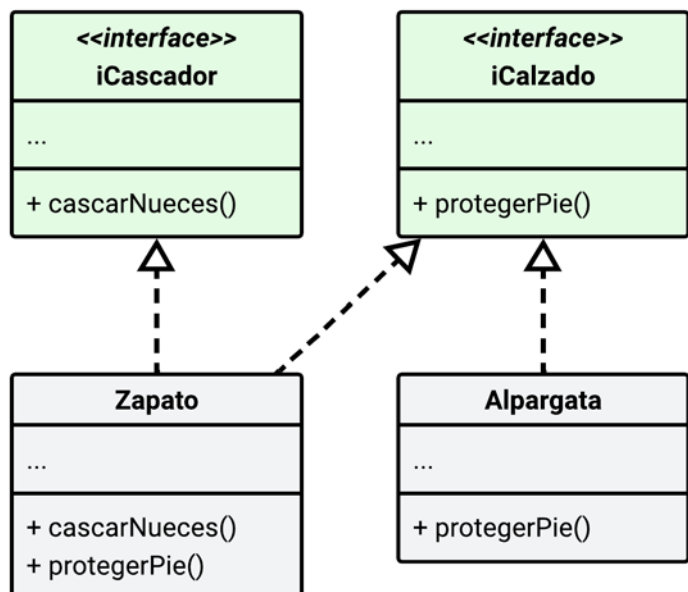
Especificación:



- ClaseA y ClaseB **implementan** la interfaz iInterfaz.

Ejemplo Práctico:

- La interfaz **iCascador** tiene un método **cascarNueces**.
- La interfaz **iCalzado** tiene un método **protegerPie**.
- La clase **zapato** implementa la interfaz **iCalzado** (un zapato puede **protegerPie**) y también **iCascador** (un zapato puede **cascarNueces**).
- La clase **alpargata** solamente implementa la interfaz **iCalzado** (un zapato puede **protegerPie**), pero no la interfaz **iCascador** (un zapato no puede **cascarNueces**).



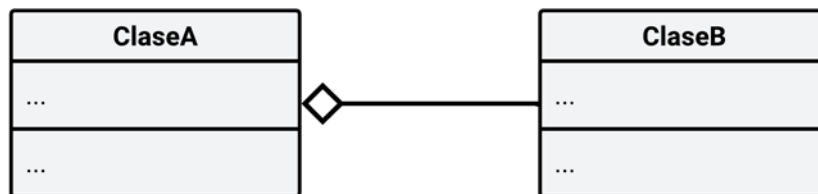
AGREGACIÓN

- Se establece entre una clase y una clase.
- Una clase agrupa varias instancias de otra clase.
- Similar a **ASOCIACIÓN**, sólo que:
 - o La relación es más fuerte, pero más débil que **COMPOSICIÓN**.
 - o Ahora la relación es *one-to-many*, ya no es *one-to-one*.
- Los ciclos de vida de ambas clases NO están atados entre sí.

Formato:

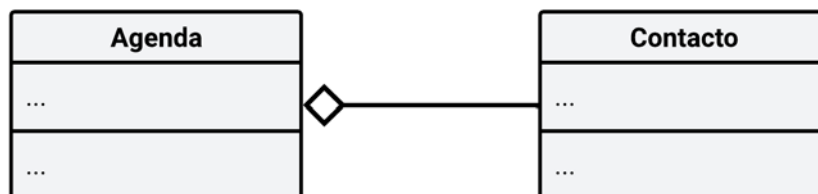
- Se representa con una línea continua, empezando con un rombo hueco.
- El sentido de la línea es unidireccional:
 - o Origen (rombo hueco): la clase que agrupa varias instancias de otra clase.
 - o Destino: la clase que es agrupada por la primera.

Especificación:



- ClaseA agrupa varios elementos del tipo ClaseB.

Ejemplo Práctico:



- La clase Agenda.
- La clase Contacto.
- Cada instancia de Agenda agrupa varias instancias de la clase Contacto.
- Los ciclos de vida de Agenda y Contacto NO están atados entre sí.
Si Agenda se va de la memoria, Contacto puede quedarse sin problemas.
Sin Agenda, cada Contacto puede existir tranquilamente.

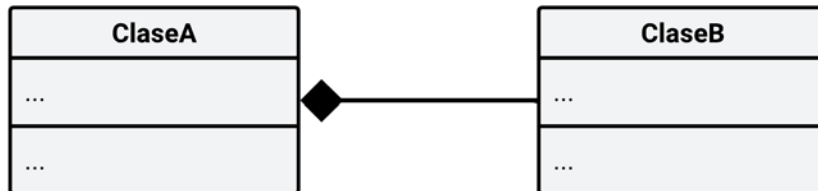
COMPOSICIÓN

- Se establece entre una clase y una clase.
- Una clase agrupa varias instancias de otra clase.
- Similar a **ASOCIACIÓN**, sólo que:
 - o La relación es más fuerte, más fuerte que **AGREGACIÓN** incluso.
 - o Ahora la relación es *one-to-many*, ya no es *one-to-one*.
- Los ciclos de vida de ambas clases están atados entre sí.

Formato:

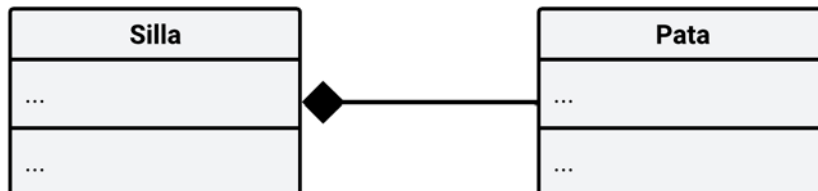
- Se representa con una línea continua, empezando con un rombo macizo.
- El sentido de la línea es unidireccional:
 - o Origen (rombo macizo): la clase que agrupa varias instancias de otra clase.
 - o Destino: la clase que es agrupada por la primera.

Especificación:



- ClaseA agrupa varios elementos del tipo ClaseB.

Ejemplo Práctico:



- La clase Silla.
- La clase Pata.
- Cada instancia de Silla agrupará a 4 instancias de la clase Pata.
- Los ciclos de vida de Silla y Pata están atados entre sí.
 - Si Silla se va de la memoria, Pata también.
 - Si no existe una Silla, no pueden existir sus 4 Patas.
 - Sin la Silla, las Patas por sí solas no tienen sentido de existencia.
 - Las Patas solamente existen si existe la Silla.